Pragya Chourasia

GitHub repository: https://github.com/pragyachourasia-05/data260-6758 (tag hw6)

DATA-260 Homework 6 - Memory for LLM Agents

Value	Definition	Mine
SID4	Last four digits of SJSU ID	6758
PORT_BASE	8000 + (SID4 mod 900)	8458
PREFIX	"s" + SID4	s6758
SEED	SID4	6758
VERIFY_SEED	260000 + SID4	266758
DOMAIN_ID	SID4 mod 8	6 - rental housing listings
Hardware		Intel Core i5-1335U, 16 GB RAM, Intel Iris Xe (CPU inference), Windows 11
Local model		llama3.2:3b (Q4_K_M) via Ollama; temperature 0, seed 6758, num_ctx 8192
Tagged commit	tag hw6 (code commit verified by verify_hw06.py)	e174eacc91aaf86b1fe01ca16bee65648e7b1a6e

Repository evidence for this homework is in `reports/hw06/`: `RUN_LOG.txt`, `raw/` (CSV/JSON results), `METRICS.md`, `AI_USE.md`, `verification.json` (from `python verify_hw06.py`), plus `memory_probes.yaml` and the MEMORY.md versions. All code extends the shared root folders (`backend/`, `src/`, `code/`, `frontend/`, `tests/`).

Part 1 - MongoDB Document API

1.1 Database and schema

Database `s6758_doc`, collection `listings` (document view of a rental listing). Validation is enforced twice: by Pydantic in the API and by a MongoDB `$jsonSchema` validator, so a document that bypasses the API is still rejected.

Field	Type	Rule
title	string	required, 1-100 characters
description	string | null	optional, max 2000
status	enum	available (default) / pending / rented
property_type	enum	apartment / house / condo / studio / townhouse
monthly_rent	number | null	optional, >= 0
available_from	date-time	required, ISO 8601
created_at / updated_at	date-time	set automatically by the server

Pydantic create schema - backend/app/doc_schemas.py, lines 11-19

```python
class ListingDocCreate(BaseModel):
    model_config = ConfigDict(extra="forbid")

    title: str = Field(..., min_length=1, max_length=100)
    description: Optional[str] = Field(None, max_length=2000)
    status: ListingStatus = "available"
    property_type: PropertyType
    monthly_rent: Optional[float] = Field(None, ge=0)
    available_from: datetime
```

MongoDB $jsonSchema validator - backend/app/mongo_db.py, lines 16-31

```python
LISTINGS_VALIDATOR = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["title", "status", "property_type", "available_from", "created_at", "
            updated_at"],
        "properties": {
            "title": {"bsonType": "string", "minLength": 1, "maxLength": 100},
            "description": {"bsonType": ["string", "null"], "maxLength": 2000},
            "status": {"enum": LISTING_STATUSES},
            "property_type": {"enum": PROPERTY_TYPES},
            "monthly_rent": {"bsonType": ["int", "long", "double", "null"], "minimum": 0},
            "available_from": {"bsonType": "date"},
            "created_at": {"bsonType": "date"},
            "updated_at": {"bsonType": "date"},
        },
    }
}
```

1.2 API, error handling and middleware

Files are split as the brief asks: `mongo_config.py` (server configuration), `mongo_db.py` (PyMongo connection), `doc_schemas.py` (Pydantic models), `routes_listings_doc.py` (route handlers), wired into the existing `backend/app/main.py` with CORS middleware and a JSON body middleware.

Method	Path	Success	Errors
POST	/api/listings	201 + document	422 invalid body, 415 non-JSON, 500 DB error
GET	/api/listings	200 list (filters ?status, ?property_type)	500
GET	/api/listings/{id}	200	400 bad ObjectId, 404 not found
PUT	/api/listings/{id}	200 (updated_at refreshed)	400 empty body / bad id, 404, 422
DELETE	/api/listings/{id}	204	400, 404

Create handler - backend/app/routes_listings_doc.py, lines 40-51

```python
@router.post("", response_model=ListingDocOut, status_code=status.HTTP_201_CREATED)
def create_listing(payload: ListingDocCreate):
    doc = payload.model_dump()
    ts = _now()
    doc["created_at"] = ts
    doc["updated_at"] = ts
    try:
        col = get_listings_collection()
        res = col.insert_one(doc)
        return _serialize(col.find_one({"_id": res.inserted_id}))
    except PyMongoError as exc:
        _db_error("creating listing", exc)
```

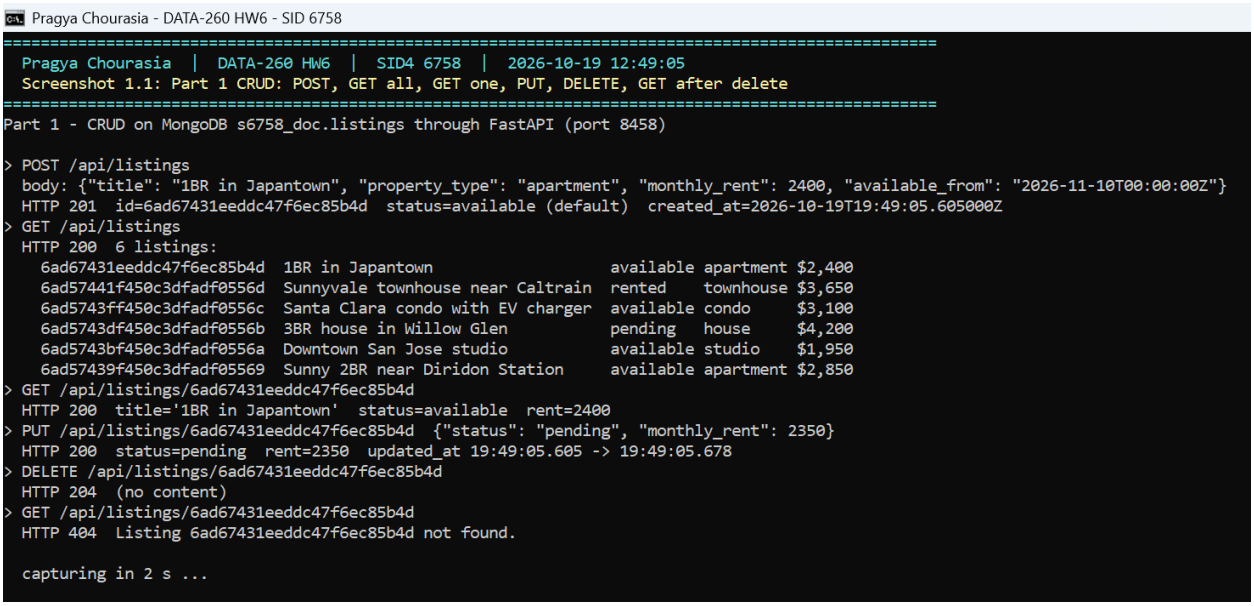
Get-by-id handler with 400/404 handling - backend/app/routes_listings_doc.py, lines 71-80

```python
@router.get("/{listing_id}", response_model=ListingDocOut)
def get_listing(listing_id: str):
    oid = _object_id(listing_id)
    try:
        doc = get_listings_collection().find_one({"_id": oid})
    except PyMongoError as exc:
        _db_error("fetching listing", exc)
    if not doc:
        raise HTTPException(status_code=404, detail=f"Listing {listing_id} not found.")
    return _serialize(doc)
```

JSON body middleware - backend/app/main.py, lines 31-40

```python
@app.middleware("http")
async def require_json_body(request: Request, call_next):
    """HW6: request bodies sent to /api/* must be JSON (FastAPI then parses them)."""
    if (
        request.url.path.startswith("/api/")
        and request.method in {"POST", "PUT", "PATCH"}
        and "application/json" not in request.headers.get("content-type", "")
    ):
        return JSONResponse(status_code=415, content={"detail": "Content-Type must be
            application/json."})
    return await call_next(request)
```

1.3 CRUD and validation evidence



Screenshot 1.1 - Terminal: POST, GET all, GET one, PUT, DELETE, then GET after delete (201, 200, 200, 200, 204, 404)

Invalid request	Expected	Message
POST without title	422	body.title: Field required
property_type = "castle"	422	Input should be 'apartment', 'house', ...
available_from = "next week", monthly_rent = "cheap"	422	invalid datetime + unable to parse number
Content-Type: text/plain	415	Content-Type must be application/json.
GET /api/listings/123	400	Invalid listing id '123'
PUT status = "sold"	422	Input should be 'available', 'pending' or 'rented'

```
Pragya Chourasia - DATA-260 HW6 - SID 6758
================================================================================================
  Pragya Chourasia  |  DATA-260 HW6  |  SID4 6758  |  2026-10-19 12:49:09
  Screenshot 1.2: Part 1 validation errors (422 / 415 / 400 / 404)
================================================================================================
Part 1 - validation and error handling (POST/PUT /api/listings)

> POST /api/listings  {"property_type": "apartment", "available_from": "2026-11-01"}
  HTTP 422  title: Field required
> POST /api/listings  {"title": "x", "property_type": "castle", "available_from": "2026-11-01"}
  HTTP 422  property_type: Input should be 'apartment', 'house', 'condo', 'studio' or 'townhouse'
> POST /api/listings  {"title": "x", "property_type": "condo", "available_from": "next week", "monthly_rent": "cheap"}
  HTTP 422  monthly_rent: Input should be a valid number, unable to parse string as a number; available_from: Input
> POST /api/listings  {"title": "<101 x's>", "property_type": "condo", "available_from": "2026-11-01"}
  HTTP 422  title: String should have at most 100 characters
> PUT /api/listings/65f000000000000000000000  {"status": "sold"}
  HTTP 422  status: Input should be 'available', 'pending' or 'rented'
> POST /api/listings  Content-Type: text/plain  body: title=x
  HTTP 415  Content-Type must be application/json.
> GET /api/listings/123
  HTTP 400  Invalid listing id '123': expected a 24-character hex ObjectId.
> GET /api/listings/65f000000000000000000000
  HTTP 404  Listing 65f000000000000000000000 not found.

  capturing in 2 s ...
```

Screenshot 1.2 - Terminal: validation and error handling - 422 for missing/invalid fields, 415 non-JSON, 400 bad id, 404 unknown id

```
Windows PowerShell   Pragya Chourasia - DATA-260
================================================================================================
  Pragya Chourasia  |  DATA-260 HW6  |  SID4 6758  |  2026-10-19 12:12:20
  Screenshot 1.3: Part 1 MongoDB shell: s6758_doc.listings with sample data + validator
================================================================================================
Part 1 - mongo shell: database s6758_doc, collection listings (sample data + validator)

> db.listings.find({}, {title:1, status:1, property_type:1, monthly_rent:1, available_from:1, created_at:1})
{"_id":{"$oid":"6ad57439f450c3dfadf05569"},"title":"Sunny 2BR near Diridon Station","status":"available","property_type"
:"apartment","monthly_rent":2850,"available_from":{"$date":"2026-11-01T00:00:00Z"},"created_at":{"$date":"2026-10-19T01:
36:56.919Z"}}
{"_id":{"$oid":"6ad5743bf450c3dfadf0556a"},"title":"Downtown San Jose studio","status":"available","property_type":"stud
io","monthly_rent":1950,"available_from":{"$date":"2026-11-15T00:00:00Z"},"created_at":{"$date":"2026-10-19T01:36:59.264
Z"}}
{"_id":{"$oid":"6ad5743df450c3dfadf0556b"},"title":"3BR house in Willow Glen","status":"pending","property_type":"house"
,"monthly_rent":4200,"available_from":{"$date":"2026-12-01T00:00:00Z"},"created_at":{"$date":"2026-10-19T01:37:01.373Z"}
}
{"_id":{"$oid":"6ad5743ff450c3dfadf0556c"},"title":"Santa Clara condo with EV charger","status":"available","property_ty
pe":"condo","monthly_rent":3100,"available_from":{"$date":"2026-11-20T00:00:00Z"},"created_at":{"$date":"2026-10-19T01:3
7:03.523Z"}}
{"_id":{"$oid":"6ad57441f450c3dfadf0556d"},"title":"Sunnyvale townhouse near Caltrain","status":"rented","property_type"
:"townhouse","monthly_rent":3650,"available_from":{"$date":"2026-10-01T00:00:00Z"},"created_at":{"$date":"2026-10-19T01:
37:05.618Z"}}

> validator required fields / enums:
required: ["title","status","property_type","available_from","created_at","updated_at"]
status enum: ["available","pending","rented"]  property_type enum: ["apartment","house","condo","studio","townhouse"]
title: {"bsonType":"string","minLength":1,"maxLength":100}

  [Enter] capture this window   [r] run again   [s] skip:
```

Screenshot 1.3 - Mongo shell: s6758_doc.listings with the sample documents and the $jsonSchema rules

Part 2 - Short-, Long-, and Episodic Memory

2.1 Collections (same database, Part 1 collection unchanged)

Collection	Fields	Written when
messages	user_id, session_id, role (user/assistant), content, created_at	every user message and reply
summaries	user_id, session_id (null for lifetime), scope ('session' | 'user'), text, created_at	every 4 user messages; lifetime every 2nd session summary
episodes	user_id, session_id, fact, importance (0..1), embedding (512 floats), created_at	up to 3 facts per user message

2.2 Endpoints

`POST /api/chat` takes `{user_id, session_id?, message}` and returns the reply plus `memory_used` (short-term count and list, session and lifetime summary, episodic facts with similarity) and `usage` (tokens and latency). `GET /api/memory/{user_id}` returns the last 16 messages, latest session and lifetime summaries, and last 20 episodes. `GET /api/aggregate/{user_id}` returns daily message counts (Pacific time) and the 5 most recent summaries.

POST /api/chat - backend/app/routes_memory.py, lines 22-29

```
22  @router.post("/chat", response_model=ChatResponse)
23  def chat(req: ChatRequest):
24      try:
25          return memory.chat_turn(req.user_id, req.message, req.session_id, req.short_term_n,
                req.policy)
26      except PyMongoError:
27          raise HTTPException(status_code=503, detail="MongoDB is unavailable.")
28      except (httpx.HTTPError, ConnectionError) as exc:
29          raise HTTPException(status_code=502, detail=f"Local model (Ollama) call failed: {exc
                }")
```

GET /api/memory/{user_id} - backend/app/routes_memory.py, lines 32-50

```python
@router.get("/memory/{user_id}")
def get_memory(user_id: str):
    try:
        messages, summaries, episodes = get_memory_collections()
        recent = list(messages.find({"user_id": user_id}).sort([("created_at", DESCENDING),
            ("_id", DESCENDING)]).limit(16))
        session_summary = summaries.find_one({"user_id": user_id, "scope": "session"}, sort=
            [("created_at", DESCENDING)])
        lifetime = summaries.find_one({"user_id": user_id, "scope": "user"}, sort=[("
            created_at", DESCENDING)])
        facts = list(episodes.find({"user_id": user_id}, {"embedding": 0}).sort("created_at"
            , DESCENDING).limit(20))
    except PyMongoError:
        raise HTTPException(status_code=503, detail="MongoDB is unavailable.")
    if not recent and not facts:
        raise HTTPException(status_code=404, detail=f"No memory stored for user '{user_id}'.
            ")
    return {
        "user_id": user_id,
        "recent_messages": [_clean(m) for m in reversed(recent)],
        "latest_session_summary": _clean(session_summary),
        "latest_lifetime_summary": _clean(lifetime),
        "episodic_facts": [_clean(f) for f in facts],
    }
```

GET /api/aggregate/{user_id} - backend/app/routes_memory.py, lines 53-74

```python
@router.get("/aggregate/{user_id}")
def aggregate(user_id: str):
    try:
        messages, summaries, _ = get_memory_collections()
        daily = list(messages.aggregate([
            {"$match": {"user_id": user_id}},
            {"$group": {
                "_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at", "
                    timezone": "America/Los_Angeles"}},
                "total": {"$sum": 1},
                "user_messages": {"$sum": {"$cond": [{"$eq": ["$role", "user"]}, 1, 0]}},
                "assistant_messages": {"$sum": {"$cond": [{"$eq": ["$role", "assistant"]}, 1
                    , 0]}},
            }},
            {"$sort": {"_id": 1}},
        ]))
        recent_summaries = list(summaries.find({"user_id": user_id}).sort("created_at",
            DESCENDING).limit(5))
    except PyMongoError:
        raise HTTPException(status_code=503, detail="MongoDB is unavailable.")
    return {
        "user_id": user_id,
        "daily_message_counts": [{"date": d.pop("_id"), **d} for d in daily],
        "recent_summaries": [_clean(s) for s in recent_summaries],
    }
```

Server steps for one turn - backend/app/memory.py, lines 365-388 (excerpt; full code in the repository)

```python
def chat_turn(user_id: str, message: str, session_id: str | None = None,
              short_term_n: int | None = None, policy: str | None = None) -> dict:
    t0 = time.perf_counter()
    session_id = session_id or f"sess-{uuid.uuid4().hex[:8]}"
    n = cfg.SHORT_TERM_N if short_term_n is None else short_term_n
    policy = policy or "none"

    msg_id = save_message(user_id, session_id, "user", message)
    window = apply_policy(load_short_term(user_id, session_id, n, exclude_id=msg_id), policy
        )
    session_summary = latest_summary(user_id, "session", session_id)
    lifetime_summary = latest_summary(user_id, "user")
    facts = retrieve_episodes(user_id, embed(message), cfg.EPISODIC_TOP_K)

    prompt, components = build_prompt(
        message, window,
        session_summary["text"] if session_summary else None,
        lifetime_summary["text"] if lifetime_summary else None,
        facts,
    )
    t_model = time.perf_counter()
    result = chat_client().complete(prompt)
    model_ms = (time.perf_counter() - t_model) * 1000
    reply = result["text"].strip()
    save_message(user_id, session_id, "assistant", reply)
```

2.3 Memory logic

Short-term window (last N messages) - backend/app/memory.py, lines 172-181

```python
def load_short_term(user_id: str, session_id: str, n: int, exclude_id=None) -> list[dict]:
    """Sliding window: the last n messages of the session, oldest first."""
    if n <= 0:
        return []
    messages, _, _ = get_memory_collections()
    query = {"user_id": user_id, "session_id": session_id}
    if exclude_id is not None:
        query["_id"] = {"$ne": exclude_id}
    docs = list(messages.find(query, {"role": 1, "content": 1}).sort([("created_at",
        DESCENDING), ("_id", DESCENDING)]).limit(n))
    return [{"role": d["role"], "content": d["content"]} for d in reversed(docs)]
```

Summarization trigger - backend/app/memory.py, lines 247-262

```python
def maybe_summarize(user_id: str, session_id: str) -> dict:
    """Summarization trigger: every SUMMARIZE_EVERY_USER_MSGS user messages."""
    messages, summaries, _ = get_memory_collections()
    user_count = messages.count_documents({"user_id": user_id, "session_id": session_id, "role": "user"})
    info = {"session_summary": False, "lifetime_summary": False, "input_tokens": 0, "output_tokens": 0}
    if user_count == 0 or user_count % cfg.SUMMARIZE_EVERY_USER_MSGS:
        return info
    r = summarize_session(user_id, session_id)
    info.update(session_summary=True, input_tokens=r["input_tokens"], output_tokens=r["output_tokens"])
    n_session_summaries = summaries.count_documents({"user_id": user_id, "scope": "session"})
    if n_session_summaries % cfg.LIFETIME_EVERY_SESSION_SUMMARIES == 0:
        r = condense_lifetime(user_id)
        info["lifetime_summary"] = True
        info["input_tokens"] += r["input_tokens"]
        info["output_tokens"] += r["output_tokens"]
    return info
```

Episode extraction (up to 3 facts, importance 0..1) - backend/app/memory.py, lines 268-297 (excerpt; full code in the repository)

```python
def extract_episodes(user_id: str, session_id: str, message: str) -> dict:
    """Extract up to 3 short facts (with importance 0..1), embed and store them."""
    _, _, episodes = get_memory_collections()
    prompt = (
        "Extract durable personal facts the USER states about themselves in this message "
        "(e.g. name, budget, move-in date, pets, workplace, must-haves, deal-breakers).\n"
        f"Message: \"{message}\"\n\n"
        'Return JSON: {"facts": [{"fact": "<one short sentence, third person>", "importance": <0..1>}]}. '
        "At most 3 facts. Questions, greetings and small talk have no facts: return {\"facts\": []}."
    )
    result = json_client().complete([{"role": "user", "content": prompt}])
    stored, skipped = [], 0
    try:
        facts = json.loads(result["text"]).get("facts", [])
    except (json.JSONDecodeError, AttributeError):
        facts = []
    existing = list(episodes.find({"user_id": user_id}, {"embedding": 1}))
    for item in facts[: cfg.MAX_FACTS_PER_MESSAGE]:
        if not isinstance(item, dict):
            continue
        fact = str(item.get("fact", "")).strip()[:300]
        if not fact:
            continue
        try:
            importance = min(1.0, max(0.0, float(item.get("importance", 0.5))))
        except (TypeError, ValueError):
            importance = 0.5
        emb = embed(fact)
        if any(cosine(emb, e["embedding"]) > 0.95 for e in existing):
            skipped += 1  # near-duplicate of a stored episode
```

Embedding (local feature hashing, 512 dims) - backend/app/memory.py, lines 113-130

```python
def _hash_embed(text: str, dim: int) -> list[float]:
    """Feature-hashing embedding: stemmed unigrams + bigrams + char trigrams.

    Deterministic and dependency-free, so retrieval is reproducible across runs.
    """
    vec = np.zeros(dim, dtype=np.float64)
    words = [_stem(w) for w in re.findall(r"[a-z0-9$]+", text.lower()) if w not in _STOP]
    features: list[tuple[str, float]] = [(w, 1.0) for w in words]
    features += [(f"{a}_{b}", 0.7) for a, b in zip(words, words[1:])]
    for w in words:
        padded = f"#{w}#"
        features += [(padded[i:i + 3], 0.25) for i in range(len(padded) - 2)]
    for feat, weight in features:
        h = int(hashlib.md5(feat.encode()).hexdigest(), 16)
        sign = 1.0 if (h >> 1) & 1 else -1.0
        vec[h % dim] += sign * weight
    norm = np.linalg.norm(vec)
    return (vec / norm).tolist() if norm else vec.tolist()
```

Top-k retrieval by cosine similarity - backend/app/memory.py, lines 147-158

```python
def retrieve_episodes(user_id: str, query_embedding: list[float], k: int) -> list[dict]:
    """Top-k episodes for this user by cosine similarity to the current message."""
    _, _, episodes = get_memory_collections()
    scored = []
    for ep in episodes.find({"user_id": user_id}, {"fact": 1, "importance": 1, "embedding":
        1, "created_at": 1}):
        scored.append({
            "fact": ep["fact"],
            "importance": ep["importance"],
            "score": round(cosine(query_embedding, ep["embedding"]), 4),
        })
    scored.sort(key=lambda e: (e["score"], e["importance"]), reverse=True)
    return [e for e in scored[:k] if e["score"] > 0]
```

Prompt assembly - backend/app/memory.py, lines 339-362

```python
def build_prompt(message: str, window: list[dict], session_summary: str | None,
                 lifetime_summary: str | None, facts: list[dict]) -> tuple[list[dict], dict]:
    """system primer + long-term summaries + short-term window + current message + episodic facts."""
    long_term = ""
    if lifetime_summary:
        long_term += f"\n\nLifetime summary of this user:\n{lifetime_summary}"
    if session_summary:
        long_term += f"\n\nSummary of this session so far:\n{session_summary}"
    episodic = ""
    if facts:
        episodic = "\n\n[Relevant facts remembered about the user]\n" + "\n".join(f"- {f['fact']}" for f in facts)

    msgs = [{"role": "system", "content": SYSTEM_PRIMER + long_term}]
    msgs += window
    msgs.append({"role": "user", "content": message + episodic})

    components = {
        "system_primer": estimate_tokens(SYSTEM_PRIMER),
        "long_term_summary": estimate_tokens(long_term),
        "short_term_window": sum(estimate_tokens(m["content"]) for m in window),
        "current_message": estimate_tokens(message),
        "episodic_facts": estimate_tokens(episodic),
    }
    return msgs, components
```

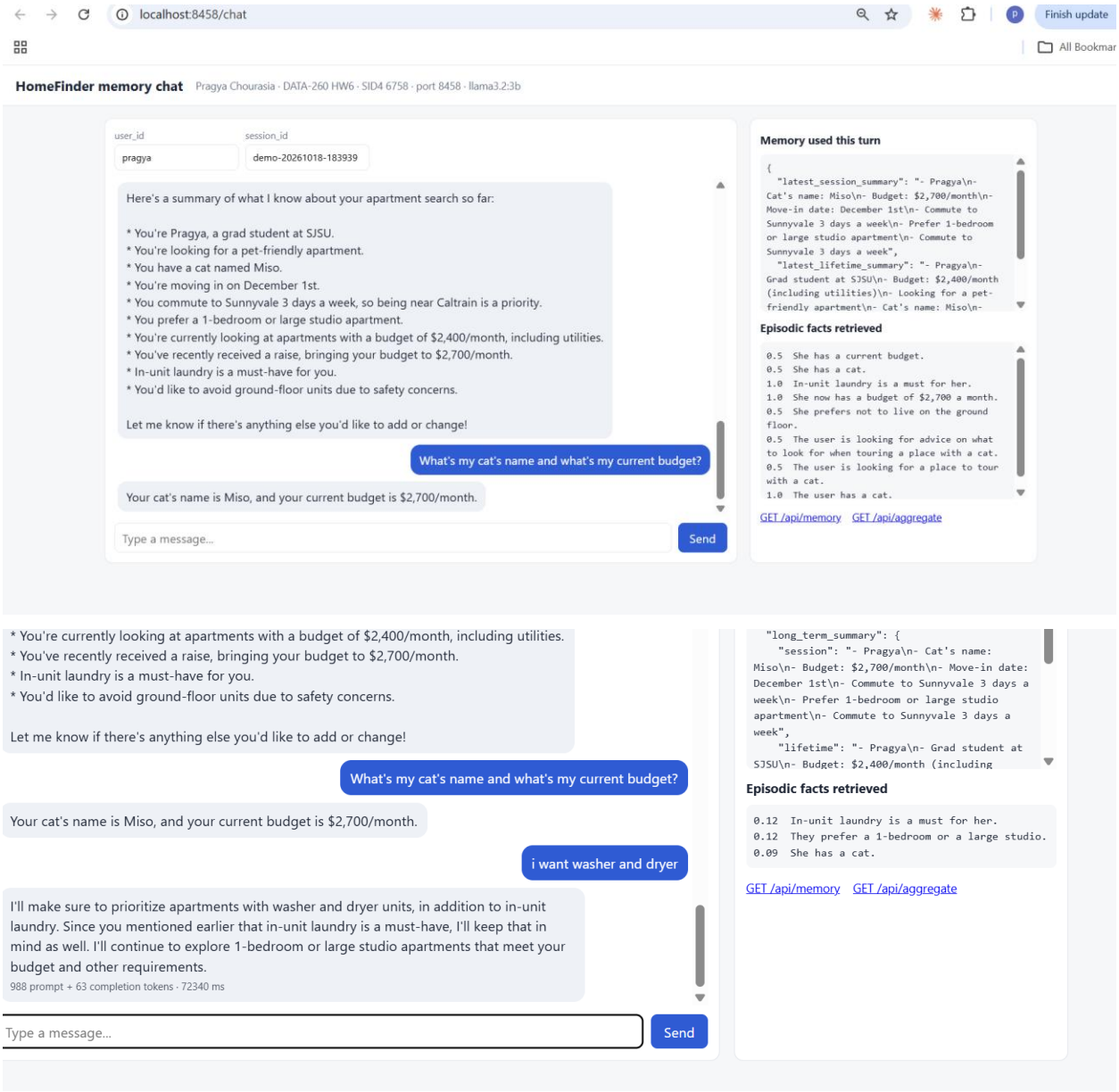
Embeddings use deterministic feature hashing (stemmed words, bigrams, character trigrams) because the local Ollama install had no embedding model; `EMBED_BACKEND=ollama` switches to an Ollama embedding model such as nomic-embed-text.

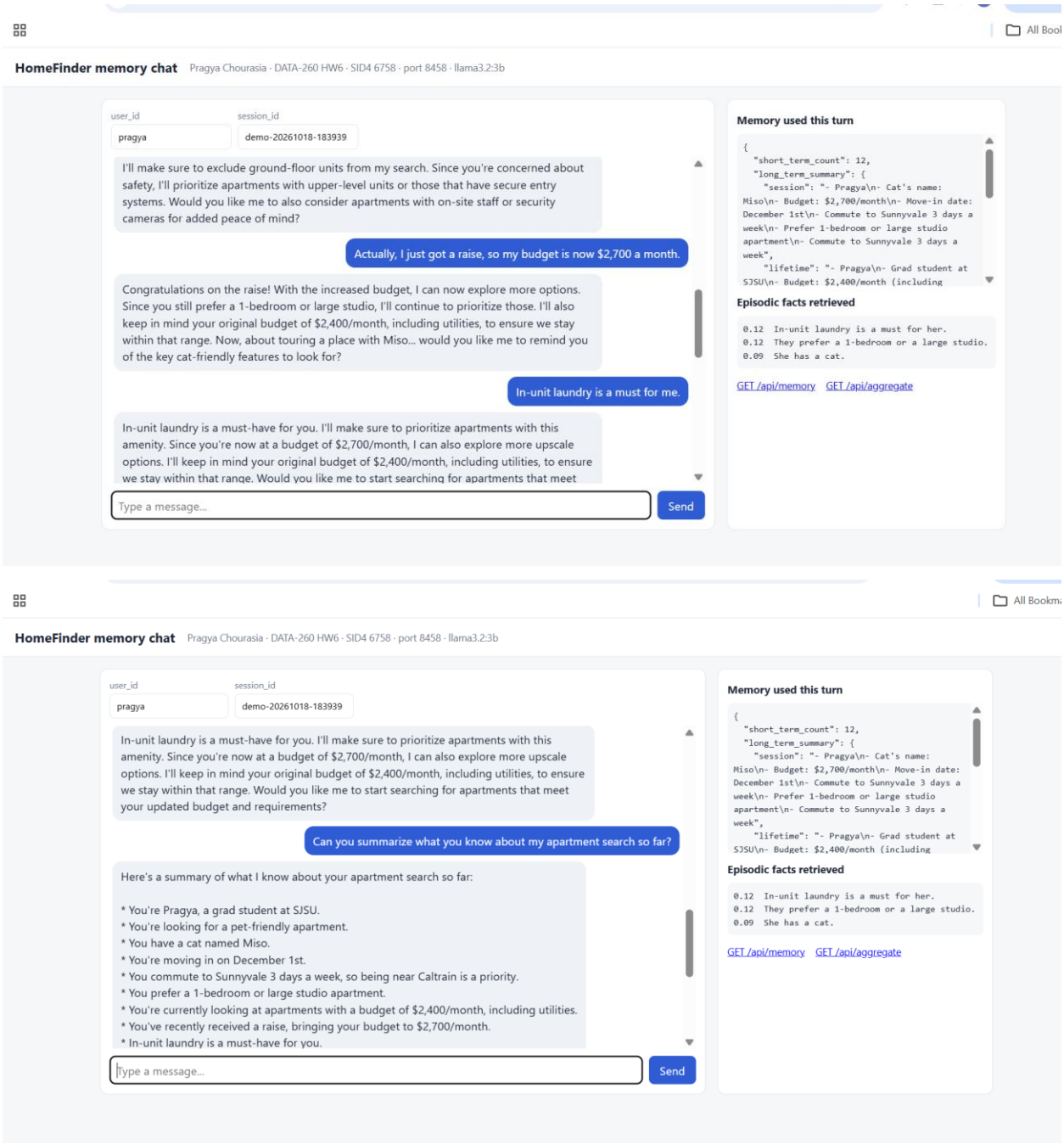
2.4 Demo session (12 messages, user pragya)

Turn	Window	Facts retrieved	New facts	Summary	Prompt tokens
1	0	0	2	-	151
2	2	1	1	-	254
3	4	3	1	-	374
4	6	3	2	session	443
5	8	3	2	-	611
6	10	3	1	-	704
7	12	3	3	-	791
8	12	3	1	session + lifetime	828
9	12	3	1	-	934
10	12	3	1	-	970
11	12	3	0	-	956
12	12	3	2	session	1046

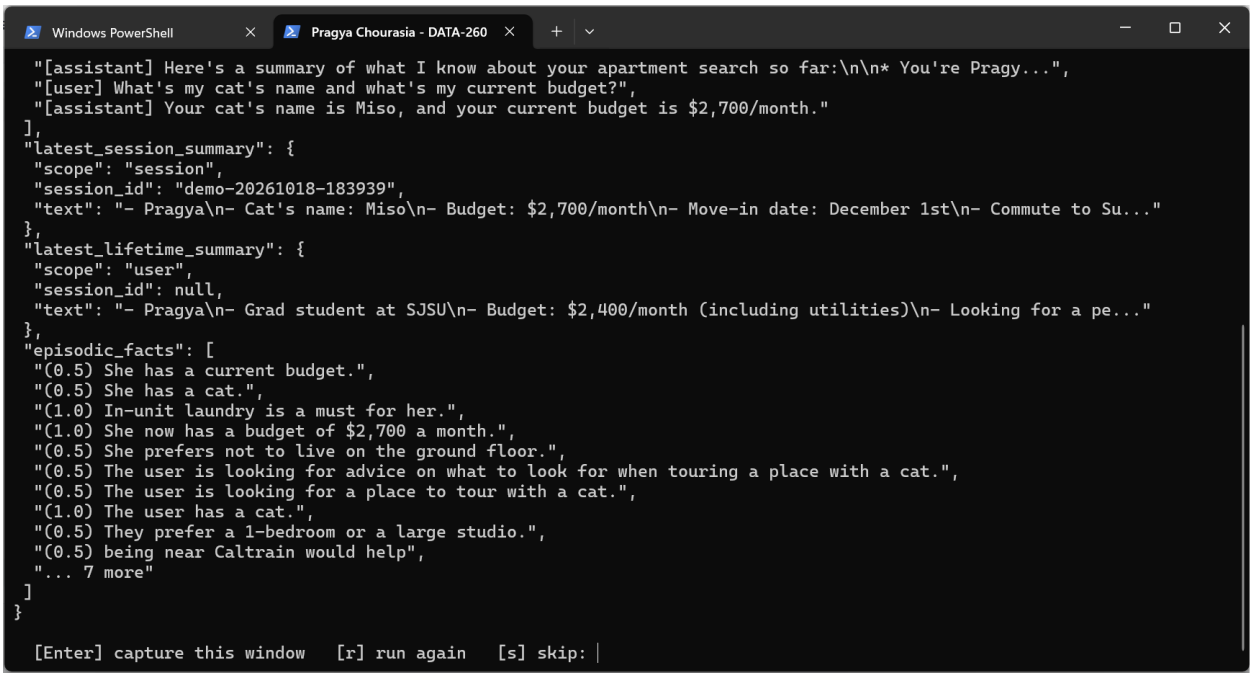
Final turn: "What's my cat's name and what's my current budget?" -> "Your cat's name is Miso, and your current budget is $2,700/month." The window grows to N=12 by turn 7, summaries fire at turns 4, 8 and 12, and the lifetime summary is built at turn 8.

Observation: the lifetime summary built at turn 8 still says "$2,400", because the user's raise to $2,700 came at turn 9 and the next lifetime refresh is due at turn 16. On turns 9-10 the old budget fact was also retrieved from episodic memory and the model mentioned both values. Episodic memory stores facts but does not mark them as superseded.

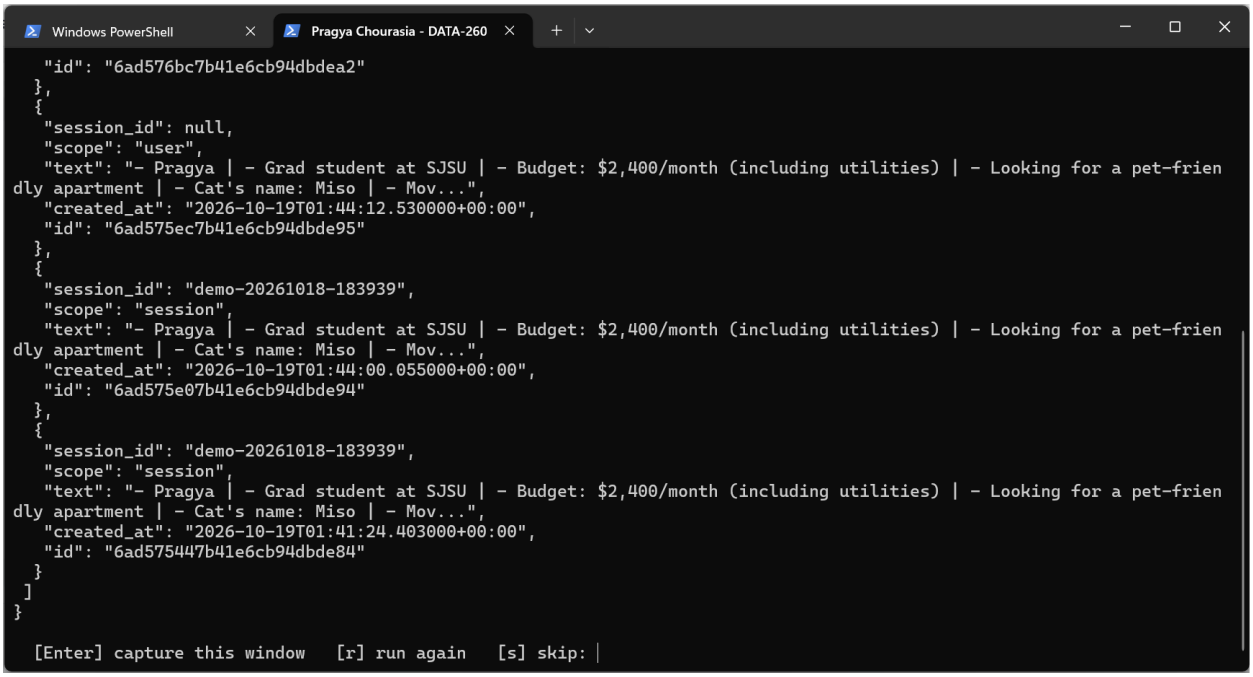

localhost:8458/chat
Finish update
All Bookmar
HomeFinder memory chat
Pragya Chourasia · DATA-260 HW6 · SID4 6758 · port 8458 · llama3.2:3b
user_id
pragya
session_id
demo-20261018-183939
Here's a summary of what I know about your apartment search so far:
* You're Pragya, a grad student at SJSU.
* You're looking for a pet-friendly apartment.
* You have a cat named Miso.
* You're moving in on December 1st.
* You commute to Sunnyvale 3 days a week, so being near Caltrain is a priority.
* You prefer a 1-bedroom or large studio apartment.
* You're currently looking at apartments with a budget of $2,400/month, including utilities.
* You've recently received a raise, bringing your budget to $2,700/month.
* In-unit laundry is a must-have for you.
* You'd like to avoid ground-floor units due to safety concerns.
Let me know if there's anything else you'd like to add or change!
What's my cat's name and what's my current budget?
Your cat's name is Miso, and your current budget is $2,700/month.
Type a message...
Send
Memory used this turn
{
"latest_session_summary": "- Pragya\n- Cat's name: Miso\n- Budget: $2,700/month\n- Move-in date: December 1st\n- Commute to Sunnyvale 3 days a week\n- Prefer 1-bedroom or large studio apartment\n- Commute to Sunnyvale 3 days a week",
"latest_lifetime_summary": "- Pragya\n- Grad student at SJSU\n- Budget: $2,400/month (including utilities)\n- Looking for a pet-friendly apartment\n- Cat's name: Miso\n-
Episodic facts retrieved
0.5 She has a current budget.
0.5 She has a cat.
1.0 In-unit laundry is a must for her.
1.0 She now has a budget of $2,700 a month.
0.5 She prefers not to live on the ground floor.
0.5 The user is looking for advice on what to look for when touring a place with a cat.
0.5 The user is looking for a place to tour with a cat.
1.0 The user has a cat.
GET /api/memory
GET /api/aggregate
* You're currently looking at apartments with a budget of $2,400/month, including utilities.
* You've recently received a raise, bringing your budget to $2,700/month.
* In-unit laundry is a must-have for you.
* You'd like to avoid ground-floor units due to safety concerns.
Let me know if there's anything else you'd like to add or change!
What's my cat's name and what's my current budget?
Your cat's name is Miso, and your current budget is $2,700/month.
i want washer and dryer
I'll make sure to prioritize apartments with washer and dryer units, in addition to in-unit laundry. Since you mentioned earlier that in-unit laundry is a must-have, I'll keep that in mind as well. I'll continue to explore 1-bedroom or large studio apartments that meet your budget and other requirements.
988 prompt + 63 completion tokens · 72340 ms
Type a message...
Send
"long_term_summary": {
"session": "- Pragya\n- Cat's name: Miso\n- Budget: $2,700/month\n- Move-in date: December 1st\n- Commute to Sunnyvale 3 days a week\n- Prefer 1-bedroom or large studio apartment\n- Commute to Sunnyvale 3 days a week",
"lifetime": "- Pragya\n- Grad student at SJSU\n- Budget: $2,400/month (including
Episodic facts retrieved
0.12 In-unit laundry is a must for her.
0.12 They prefer a 1-bedroom or a large studio.
0.09 She has a cat.
GET /api/memory
GET /api/aggregate



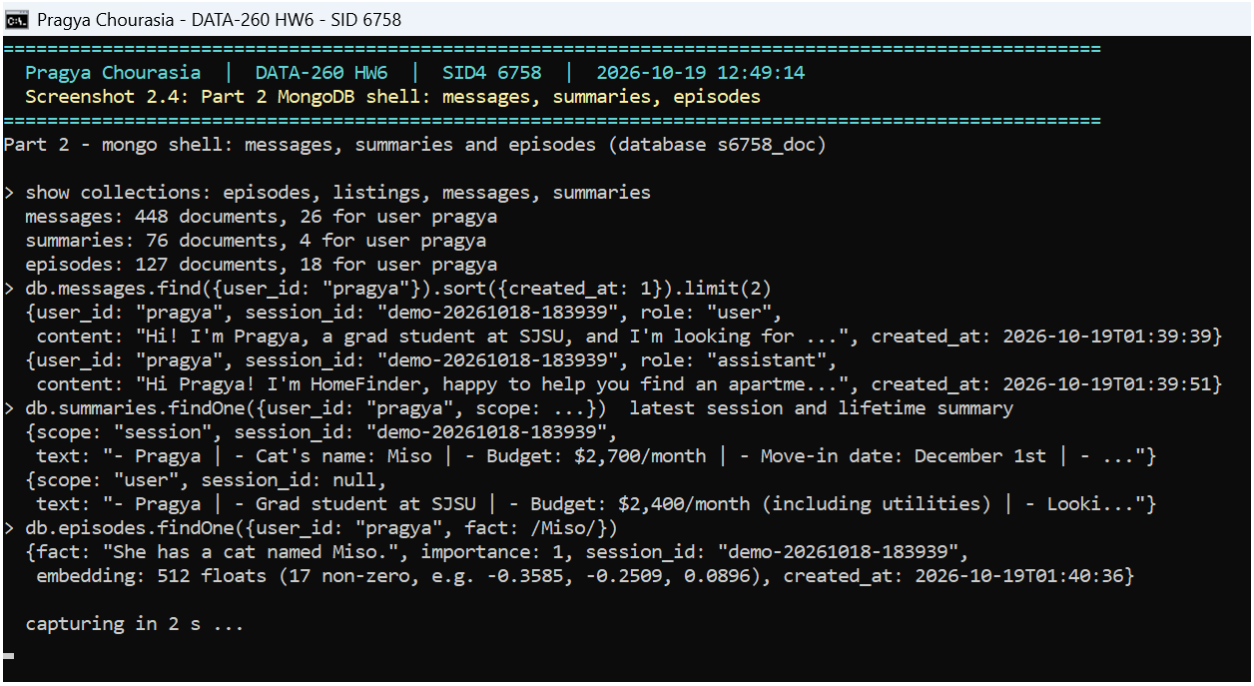
Screenshot 2.1 - Browser: http://localhost:8458/chat - stored conversation, a live turn with token count and latency, and the Memory used panel (short_term_count, summaries, episodic facts with similarity scores)



Screenshot 2.2 - Terminal: JSON result of GET /api/memory/pragya (message text shortened to fit; full JSON in raw/part2_memory.json)



Screenshot 2.3 - Terminal: JSON result of GET /api/aggregate/pragya



Screenshot 2.4 - Mongo shell: messages, summaries and episodes collections with documents

Part 3 - What Memory Costs

3.1 Usage logging

Every `/api/chat` request appends one JSON line to `raw/chat_usage.jsonl` and returns the same `usage` object: prompt and completion tokens (Ollama `prompt_eval_count` / `eval_count`), estimated tokens for the system primer, long-term summary, short-term window, current message and episodic facts, the remaining template overhead, the tokens spent on extraction and summarization, and request / model / extraction / summarization latency. Per-turn rows for the experiment are in `raw/part3_turns.csv`.

3.2 Memory probes

`memory_probes.yaml` was written before the runs. Each probe targets a fact planted at a known message so the window sizes differ in what they can see verbatim (N counts user and assistant messages: N=4 sees user messages 29-30, N=12 sees 25-30, N=30 sees 16-30).

Probe	Question	Planted at msg	Required for full credit
P1	What kind of car do I drive and what does it need where I live?	27	Tesla / Model 3 + charging
P2	Who visits me every month, and what do I need because of that?	22	Riya + sister + sofa bed/space
P3	What is my current monthly budget?	2, changed at 20	$2,700; stating $2,400 as well costs 1 point
P4	What is my cat's name?	3	Miso
P5	On what date do I need to move in, and why that date?	5	December 1 + lease ends

Score	Rubric
5	Every required element present, no outdated/wrong value
4	Every required element present, but an outdated value is also stated
3	At least half of the required elements
2	Fewer than half, or only a wrong/outdated value
1	No required element ("I don't know" or unrelated)

Deterministic scorer - backend/run_hw06_memory_cost.py, lines 69-84

```python
def score_probe(reply: str, probe: dict) -> dict:
    """Deterministic 1-5 score following the rubric in memory_probes.yaml."""
    text = reply.lower()
    required = probe["required"]
    hits = [any(_contains(text, s) for s in group) for group in required]
    forbidden = [s for s in probe.get("forbidden", []) if _contains(text, s)]
    n_hit = sum(hits)
    if n_hit == len(required):
        score = 4 if forbidden else 5
    elif n_hit >= len(required) / 2:
        score = 3
    elif n_hit > 0 or forbidden:
        score = 2
    else:
        score = 1
    return {"score": score, "required_hits": hits, "forbidden_found": forbidden}
```

3.3 Short-term memory sizes

The same 30-message conversation was replayed for each setting as a fresh user, then the five probes were asked. Summaries and episodic memory stay on in every setting; only the window size changes. Means are over the 30 conversation turns; latency is the whole `/api/chat` request on CPU.

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	501.2	30,183	3.4
SHORT_TERM_N=12	1,001.6	58,552	3.8
SHORT_TERM_N=30	1,750.4	71,053	4.6
Eviction/compression policy (N=30 + compress_assistant)	820.0	33,276	4.6

Setting	Median latency (ms)	Model-call latency (ms)	Completion tokens	P1-P5 scores
SHORT_TERM_N=4	25,090	17,923	83.8	3, 1, 5, 5, 3
SHORT_TERM_N=12	61,827	43,496	116.4	5, 2, 4, 5, 3
SHORT_TERM_N=30	83,916	56,770	111.2	5, 5, 5, 5, 3
Eviction/compression policy	36,552	22,323	51.3	5, 5, 5, 5, 3

3.4 Memory-saving policy: compress_assistant

Compression policy - backend/app/memory.py, lines 318-333

```python
def apply_policy(window: list[dict], policy: str) -> list[dict]:
    """compress_assistant: keep user turns verbatim, shrink older assistant turns.

    Facts live in the user's messages; the assistant's long replies are mostly
    restatements, so all but the last two window messages are cut to their
    first sentence (<= 25 words).
    """
    if policy != "compress_assistant":
        return window
    out = []
    for i, m in enumerate(window):
        if m["role"] == "assistant" and i < len(window) - 2:
            out.append({"role": "assistant", "content": _first_sentence(m["content"])})
        else:
            out.append(m)
    return out
```

	N=30 baseline	N=30 + compress_assistant	Change
Mean prompt tokens/turn	1,750.4	820.0	-930.4 (53.2% saved)
Mean latency (ms)	71,053	33,276	-37,777
Probe quality	4.6	4.6	+0.0

3.5 Cost estimate (10,000 conversations x 30 turns)

Input $1 per million tokens, output $5 per million tokens. "Chat" counts the reply calls only; "with upkeep" adds the extraction and summarization calls each turn makes, which are part of what memory costs.

Setting	Input tok/conv	Output tok/conv	Chat cost	With upkeep	Quality
SHORT_TERM_N=4	15,037	2,515	$276.12	$419.83	3.4
SHORT_TERM_N=12	30,047	3,491	$475.02	$626.65	3.8
SHORT_TERM_N=30	52,513	3,336	$691.93	$854.33	4.6
Eviction/compression policy	24,600	1,538	$322.90	$450.02	4.6

Best: Eviction/compression policy - $450.02 (chat $322.90). **Worst: SHORT_TERM_N=30** - $854.33 (chat $691.93).

Selection: quality first, then cost. Best is the cheapest setting among those with the top probe score; worst is the most expensive setting, which here buys no quality over the best one. The cheapest settings (small N) are not best because they lose window-only facts such as P2.

3.6 Analysis

- Prompt size grows with the window: 501 -> 1002 -> 1750 tokens per turn for N = 4, 12, 30. The short-term window is the dominant prompt component at N=30 (~1523 estimated tokens), while summaries (~141) and episodic facts (~44) stay small in every setting.
- Quality is decided by which facts only the window holds. P2 (sister Riya, message 22) was answered only by the settings whose window still contained message 22 (scores: N=4 1, N=12 2, N=30 5); summaries and episodic facts carried the early facts (cat name, budget) for every setting.
- P5 scored 3 in every setting: the move-in date survived, but no memory layer kept the reason ("my lease ends"). Summaries and extracted facts compress away the why, so reasons need to be extracted explicitly if they matter.
- compress_assistant cut prompt tokens by 53% versus N=30 with a quality change of +0.0: the facts live in user messages, and long assistant replies are mostly restatement. It also shortened replies (completion 51 vs 111 tokens), because the model imitates the length of the earlier replies it sees.
- Memory upkeep is not free: extraction and summarization add $127-$162 per 10,000 conversations (+23% to +52% on top of the chat calls). That part barely depends on the window size, so it matters most for the cheap small-window settings.
- Latency is CPU-bound and noisy on this laptop (Ollama on an i5 with ~1 GB free RAM); token counts are the reliable comparison. Runs interrupted by Windows standby were rerun with the PC kept awake and the affected runs are kept in raw/part3_run2_standby/.
- Each setting was run once, so single probe scores can move by a point between runs: in the standby-affected N=4 run P1 scored 5, in the rerun 3 (the reply named the Tesla but not the charging). The ordering of the settings did not change.

Part 4 - File-Based Memory and Consolidation

4.1 AGENT.md (final)

Expanded from HW1: 15 instructions covering all five categories (project description, technology stack, coding conventions, testing approach, prohibited actions); the HW1 code-review rules are kept as a scoped mode.

AGENT.md - AGENT.md, lines 1-33

```
# Agent Instructions

Durable instructions for any agent working in this repository (data260-6758).
Facts learned from conversations live in MEMORY.md, not here.

## Project description
- This repository is a rental-housing listings application for DATA-260: a FastAPI backend on port 8458, a React frontend, MySQL database `s6758_rel`, MongoDB database `s6758_doc`, and a local-model assistant with short-term, long-term and episodic memory.
- Shared code lives in root-level folders (`backend/`, `frontend/`, `src/`, `rag/`); every homework extends these folders instead of copying them.

## Technology stack
- Python 3.12, FastAPI, Pydantic v2, SQLAlchemy (MySQL) and PyMongo (MongoDB).
- All model calls go through `src/model_client.ModelClient` to the local Ollama model `llama3.2:3b`; no hosted or paid LLM APIs.

## Coding conventions
- Function names use snake_case; every parameter and the return value have type hints; every public function has a docstring.
- Validate inputs explicitly and raise `ValueError` with a clear message for bad input; never fail silently or return a magic value.
- Library code does not call `print()`; it returns values and lets the caller decide what to show.
- Configuration comes from environment variables or `backend/.env`; never hard-code passwords, API keys or tokens.

## Testing approach
- Every new function gets pytest tests in a `test_*.py` file: at least one normal case and one invalid input that must raise.
- Tests run offline: no network, database or model calls.

## Prohibited actions
- Never commit `.env` files, credentials, or the `venv/` folder.
- Never delete or overwrite MongoDB/MySQL data outside a test or an explicit user request.
- Never invent listings, prices, addresses or owner contact details.

## Code review mode (from HW1)
When the user asks you to review code, review it for correctness, bugs, security concerns, missing tests and maintainability issues, and follow these output rules:
- Your response must contain bullet points only; every line must begin with "- ".
- Do not write paragraphs, headings, introductions, or conclusions.
- If you find no issues, respond exactly: "- No issues found."
```

4.2 File loading, prompt construction and consolidation checks

Load both files - src/file_memory.py, lines 53-57

```python
def load_memory_files(agent_path: Path = AGENT_PATH, memory_path: Path = MEMORY_PATH) -> tuple[str, str]:
    """Return (AGENT.md text, MEMORY.md text); a missing file reads as empty."""
    agent = agent_path.read_text(encoding="utf-8") if agent_path.exists() else ""
    memory = memory_path.read_text(encoding="utf-8") if memory_path.exists() else ""
    return agent, memory
```

Add them to the system prompt - src/file_memory.py, lines 60-68

```python
def build_system_prompt(agent_text: str, memory_text: str) -> str:
    """AGENT.md instructions first, then the durable facts from MEMORY.md."""
    facts = "\n".join(line for line in memory_text.splitlines() if is_content_line(line))
    return (
        f"{agent_text.strip()}\n\n"
        "# Long-term memory (from MEMORY.md)\n"
        "Facts learned in earlier conversations. Treat them as true unless the user corrects them.\n"
        f"{facts or '(no saved facts yet)'}"
    )
```

Read the last-consolidated timestamp - src/file_memory.py, lines 85-94

```python
def read_last_consolidated(memory_text: str) -> datetime | None:
    """Parse the <!-- last_consolidated: ISO --> marker; None if missing or malformed."""
    match = _STAMP_RE.search(memory_text)
    if not match:
        return None
    try:
        stamp = datetime.fromisoformat(match.group(1).replace("Z", "+00:00"))
    except ValueError:
        return None
    return stamp if stamp.tzinfo else stamp.replace(tzinfo=timezone.utc)
```

When consolidation is required (> 40 lines or > 24 h) - src/file_memory.py, lines 97-109

```python
def needs_consolidation(memory_text: str, now: datetime | None = None) -> tuple[bool, list[str]]:
    """Consolidate when > 40 content lines or the last consolidation is > 24 h old (or missing)."""
    now = now or datetime.now(timezone.utc)
    reasons = []
    n = count_memory_lines(memory_text)
    if n > MAX_MEMORY_LINES:
        reasons.append(f"{n} non-empty, non-comment lines > {MAX_MEMORY_LINES}")
    stamp = read_last_consolidated(memory_text)
    if stamp is None:
        reasons.append("no last_consolidated timestamp")
    elif now - stamp > MAX_AGE:
        reasons.append(f"last consolidated {(now - stamp).total_seconds() / 3600:.1f} h ago (> 24 h)")
    return bool(reasons), reasons
```

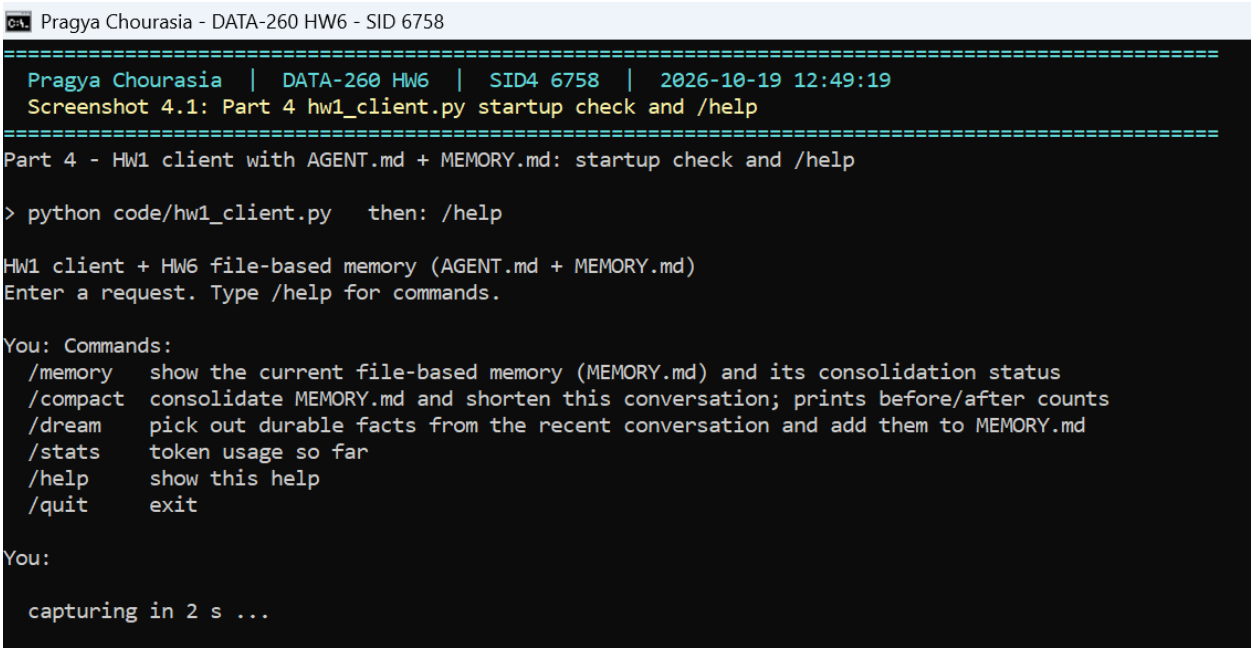
4.3 Commands

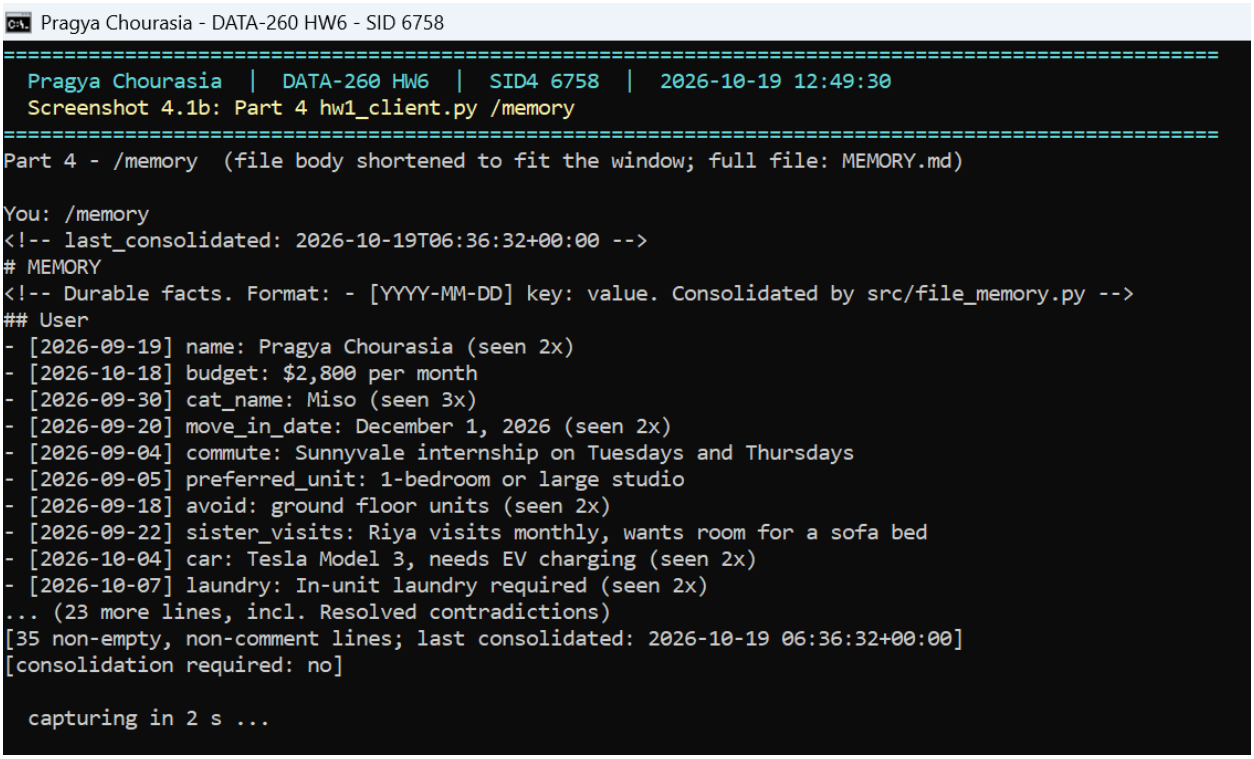
code/hw1_client.py help text - code/hw1_client.py, lines 20-26

```
20  HELP = """Commands:
21    /memory   show the current file-based memory (MEMORY.md) and its consolidation status
22    /compact  consolidate MEMORY.md and shorten this conversation; prints before/after counts
23    /dream    pick out durable facts from the recent conversation and add them to MEMORY.md
24    /stats    token usage so far
25    /help     show this help
26    /quit     exit"""
```

/compact handler - code/hw1_client.py, lines 92-103

```
 92          if user_text == "/compact":
 93              result = compact(conversation, client)
 94              refresh_system_prompt(conversation)
 95              print(f"Conversation messages: {result['messages_before']} -> {result['
                     messages_after']}")
 96              print(f"MEMORY.md lines (non-empty, non-comment): {result['memory_lines_before']
                     } -> {result['memory_lines_after']}")
 97              print(f"MEMORY.md facts: {result['facts_before']} -> {result['facts_after']}")
 98              print(f"Duplicates merged: {len(result['duplicates_merged'])}")
 99              for c in result["contradictions"]:
100                  print(f"  contradiction {c['key']}: kept \"{c['kept']}\" over \"{c['dropped'
                         ]}\" ({c['rule']})")
101              for r in result["needs_review"]:
102                  print(f"  NEEDS REVIEW {r['key']}: {r['reason']}")
103              continue
```





Screenshot 4.1 - Terminal: hw1_client.py /help, then /memory with the consolidation status

4.4 Consolidation test (55 -> 36 lines)

`reports/hw06/memory/MEMORY_original.md` has 55 lines (50 non-comment) with 17 duplicated facts (some differing only in case or punctuation, e.g. "ground-floor" vs "ground floor") and five contradictions. Rules: duplicates merge into one line with "(seen Nx)"; for a contradiction the newest dated value wins unless it is hedged ("?", "maybe"); every losing value is written to "Resolved

contradictions" (nothing is deleted silently); a reviewer pass flags a resolution as likely wrong when the dropped value was stated more often than the winner.

Contradiction resolution + review flag - src/file_memory.py, lines 215-232

```
215         latest = {v: max(fs, key=lambda f: (f.day, f.line_no)) for v, fs in variants.items()
                }
216         ordered = sorted(latest.values(), key=lambda f: (f.day, f.line_no), reverse=True)
217         confident = [f for f in ordered if not f.hedged]
218         winner = confident[0] if confident else ordered[0]
219         rule = "newest value" if winner is ordered[0] else "newer value is hedged/uncertain,
                kept newest confident value"
220         kept.append((winner, weight[winner.norm_value], first_line))
221         for loser in ordered:
222             if loser is winner:
223                 continue
224             count_l, count_w = weight[loser.norm_value], weight[winner.norm_value]
225             entry = {"key": winner.norm_key, "kept": winner.value, "kept_date": winner.day,
                    "kept_count": count_w,
226                      "dropped": loser.value, "dropped_date": loser.day, "dropped_count":
                             count_l, "rule": rule}
227             report.contradictions.append(entry)
228             new_history.append(f'- {winner.norm_key}: kept "{winner.value}" ({winner.day}
                    ) over '
229                                f'"{loser.value}" ({loser.day}) - {rule}')
230             if count_l > count_w:
231                 report.needs_review.append({**entry, "reason": f'it was stated {count_w}
                        x while the dropped '
232                                                                 f'"{loser.value}" was stated
                                                                     {count_l}x'})
```

Contradiction	Values	Resolution	Correct?
budget	$2,400 (09-02) vs $2,700 (10-10, 10-11)	kept $2,700 - newest	Yes
backend_port	8000 (08-28) vs 8458 (x2)	kept 8458 - newest	Yes
preferred_model	qwen3:8b vs llama3.2:3b (x2)	kept llama3.2:3b - newest	Yes
cat_name	Miso (x3) vs "Mochi? (maybe misheard)" (newest)	kept Miso - newer value is hedged	Yes
mongo_db	s6758_doc (x2) vs study_planner_demo (newest, x1)	kept study_planner_demo - newest; flagged "Needs review"	No - resolved incorrectly

Incorrect resolution identified: `mongo_db` - recency picked the class-demo database name `study_planner_demo`, which had been copied in once, over `s6758_doc`, which had been confirmed twice. The reviewer pass flagged it in "Needs review" and it was corrected by a recorded human review (`MEMORY_final_reviewed.md`). The same heuristic also produced a false positive later: a real budget increase to $2,800 was flagged because $2,700 had been stated twice, so the flags need a human decision.

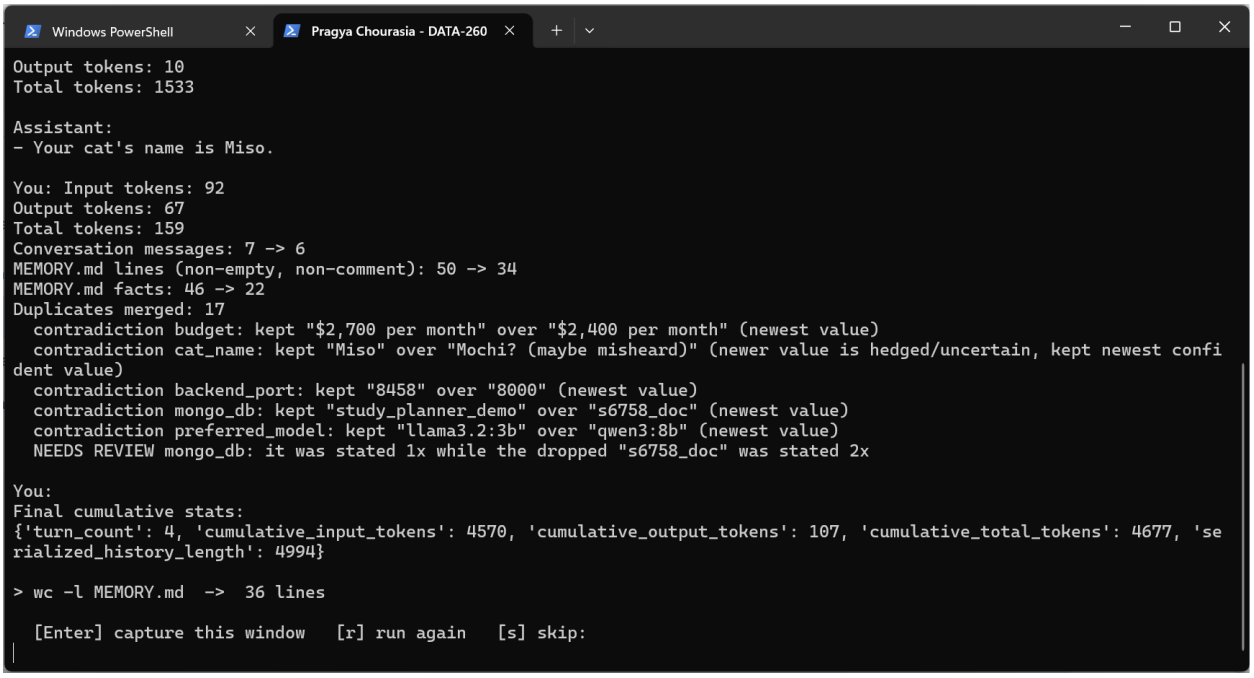
/compact output (`raw/part4_compact_session.txt`):

hw1_client.py /compact

```
Conversation messages: 7 -> 6
MEMORY.md lines (non-empty, non-comment): 50 -> 34
```

```
MEMORY.md facts: 46 -> 22
Duplicates merged: 17
  contradiction budget: kept "$2,700 per month" over "$2,400 per month" (newest value)
  contradiction cat_name: kept "Miso" over "Mochi? (maybe misheard)" (newer value is
hedged/uncertain, kept newest confident value)
  contradiction backend_port: kept "8458" over "8000" (newest value)
  contradiction mongo_db: kept "study_planner_demo" over "s6758_doc" (newest value)
  contradiction preferred_model: kept "llama3.2:3b" over "qwen3:8b" (newest value)
  NEEDS REVIEW mongo_db: it was stated 1x while the dropped "s6758_doc" was stated 2x

You:
```



Screenshot 4.2 - Terminal: /compact on the 55-line MEMORY.md with before/after counts

Before consolidation (55 lines) - reports/hw06/memory/MEMORY_original.md, lines 1-34 (part 1 of 2)

```
<!-- last_consolidated: 2026-10-01T09:00:00+00:00 -->
# MEMORY
<!-- Durable facts learned from earlier conversations. Format: - [YYYY-MM-DD] key: value -->

## User
- [2026-09-02] name: Pragya Chourasia
- [2026-09-02] budget: $2,400 per month
- [2026-09-02] cat_name: Miso
- [2026-09-03] move_in_date: December 1, 2026
- [2026-09-04] commute: Sunnyvale internship on Tuesdays and Thursdays
- [2026-09-05] preferred_unit: 1-bedroom or large studio
- [2026-09-05] avoid: ground-floor units
- [2026-09-15] cat_name: miso.
- [2026-09-18] avoid: ground floor units
- [2026-09-19] name: Pragya Chourasia
- [2026-09-20] move_in_date: December 1, 2026
- [2026-09-22] sister_visits: Riya visits monthly, wants room for a sofa bed
- [2026-09-30] Cat name: Miso
- [2026-10-02] car: Tesla Model 3, needs EV charging
- [2026-10-04] car: Tesla Model 3, needs EV charging
- [2026-10-05] laundry: in-unit laundry required
- [2026-10-07] laundry: In-unit laundry required
- [2026-10-10] budget: $2,700 per month
- [2026-10-11] budget: $2,700 per month
- [2026-10-12] cat_name: Mochi? (maybe misheard)

## Project
- [2026-08-28] repo: data260-6758
- [2026-08-28] sid4: 6758
- [2026-08-28] backend_port: 8000
- [2026-08-30] frontend: React + Vite on port 5173
- [2026-09-01] sid4: 6758
- [2026-09-01] relational_db: MySQL s6758_rel on port 3307
- [2026-09-10] backend_port: 8458
```

Before consolidation (55 lines) - reports/hw06/memory/MEMORY_original.md, lines 35-55 (part 2 of 2)

```
- [2026-09-12] backend_port: 8458
- [2026-09-14] frontend: React + Vite on port 5173
- [2026-09-25] domain: rental housing listings
- [2026-09-26] domain: Rental housing listings
- [2026-10-01] collaborators: Sbnikitha, supriyaselvanganesan
- [2026-10-08] collaborators: Sbnikitha, supriyaselvanganesan
- [2026-10-16] mongo_db: s6758_doc
- [2026-10-17] mongo_db: s6758_doc
- [2026-10-18] mongo_db: study_planner_demo

## Environment
- [2026-09-05] preferred_model: qwen3:8b
- [2026-09-05] ollama_url: http://localhost:11434
- [2026-09-06] ollama_url: http://localhost:11434
- [2026-09-07] os: Windows 11 with PowerShell and Git Bash
- [2026-09-07] hardware: Intel i5-1335U, 16 GB RAM, no GPU
- [2026-09-08] os: Windows 11 with PowerShell and Git Bash
- [2026-10-01] preferred_model: llama3.2:3b
- [2026-10-03] preferred_model: llama3.2:3b
- [2026-10-04] hardware: Intel i5-1335U, 16 GB RAM, no GPU
- [2026-10-09] ollama_url: http://localhost:11434
```

After /compact (36 lines) - reports/hw06/memory/MEMORY_consolidated.md, lines 1-34 (part 1 of 2)

```
<!-- last_consolidated: 2026-10-19T06:30:52+00:00 -->
# MEMORY
<!-- Durable facts. Format: - [YYYY-MM-DD] key: value. Consolidated by src/file_memory.py
    -->
## User
- [2026-09-19] name: Pragya Chourasia (seen 2x)
- [2026-10-11] budget: $2,700 per month (seen 2x)
- [2026-09-30] cat_name: Miso (seen 3x)
- [2026-09-20] move_in_date: December 1, 2026 (seen 2x)
- [2026-09-04] commute: Sunnyvale internship on Tuesdays and Thursdays
- [2026-09-05] preferred_unit: 1-bedroom or large studio
- [2026-09-18] avoid: ground floor units (seen 2x)
- [2026-09-22] sister_visits: Riya visits monthly, wants room for a sofa bed
- [2026-10-04] car: Tesla Model 3, needs EV charging (seen 2x)
- [2026-10-07] laundry: In-unit laundry required (seen 2x)
## Project
- [2026-08-28] repo: data260-6758
- [2026-09-01] sid4: 6758 (seen 2x)
- [2026-09-12] backend_port: 8458 (seen 2x)
- [2026-09-14] frontend: React + Vite on port 5173 (seen 2x)
- [2026-09-01] relational_db: MySQL s6758_rel on port 3307
- [2026-09-26] domain: Rental housing listings (seen 2x)
- [2026-10-08] collaborators: Sbnikitha, supriyaselvanganesan (seen 2x)
- [2026-10-18] mongo_db: study_planner_demo
## Environment
- [2026-10-03] preferred_model: llama3.2:3b (seen 2x)
- [2026-10-09] ollama_url: http://localhost:11434 (seen 3x)
- [2026-09-08] os: Windows 11 with PowerShell and Git Bash (seen 2x)
- [2026-10-04] hardware: Intel i5-1335U, 16 GB RAM, no GPU (seen 2x)
## Resolved contradictions
- budget: kept "$2,700 per month" (2026-10-11) over "$2,400 per month" (2026-09-02) - newest
    value
- cat_name: kept "Miso" (2026-09-30) over "Mochi? (maybe misheard)" (2026-10-12) - newer
    value is hedged/uncertain, kept newest confident value
- backend_port: kept "8458" (2026-09-12) over "8000" (2026-08-28) - newest value
- mongo_db: kept "study_planner_demo" (2026-10-18) over "s6758_doc" (2026-10-17) - newest
    value
- preferred_model: kept "llama3.2:3b" (2026-10-03) over "qwen3:8b" (2026-09-05) - newest
    value
```

After /compact (36 lines) - reports/hw06/memory/MEMORY_consolidated.md, lines 35-36 (part 2 of 2)

```
## Needs review
- mongo_db: kept "study_planner_demo" by recency, but it was stated 1x while the dropped
    "s6758_doc" was stated 2x - likely resolved incorrectly, confirm with the user
```

4.5 Verifying a generated function against AGENT.md

`code/generate_with_agent_md.py` asked llama3.2:3b, with AGENT.md + MEMORY.md as the system prompt, for `parse_monthly_rent` and its tests. `code/check_agent_md_rules.py` checks each AGENT.md rule separately (AST checks plus running the tests).

AGENT.md rule	Generated	Corrected
Coding conventions: snake_case function names	PASS	PASS

AGENT.md rule	Generated	Corrected
Coding conventions: type hints on every parameter and return value	PASS	PASS
Coding conventions: docstring on every public function	PASS	PASS
Coding conventions: invalid input raises ValueError	PASS	PASS
Coding conventions: no print() in library code	PASS	PASS
Coding conventions / prohibited: no hard-coded secrets	PASS	PASS
Technology stack / testing: no network, database or model imports	PASS	PASS
Testing approach: test_*.py file exists	PASS	PASS
Testing approach: at least one normal case and one invalid-input case	PASS	PASS
Testing approach: tests pass offline	FAIL	PASS

The generated code followed every style rule but failed its own first example ("$2,400/mo" raised ValueError because commas and "/mo" were never removed) and one generated test expected "$2,400" to be rejected. The corrected version uses one regex for sign, $, digits with commas/decimals and a k suffix; the checker reports 10/10.

Corrected function - src/generated/rent_parser.py, lines 1-29

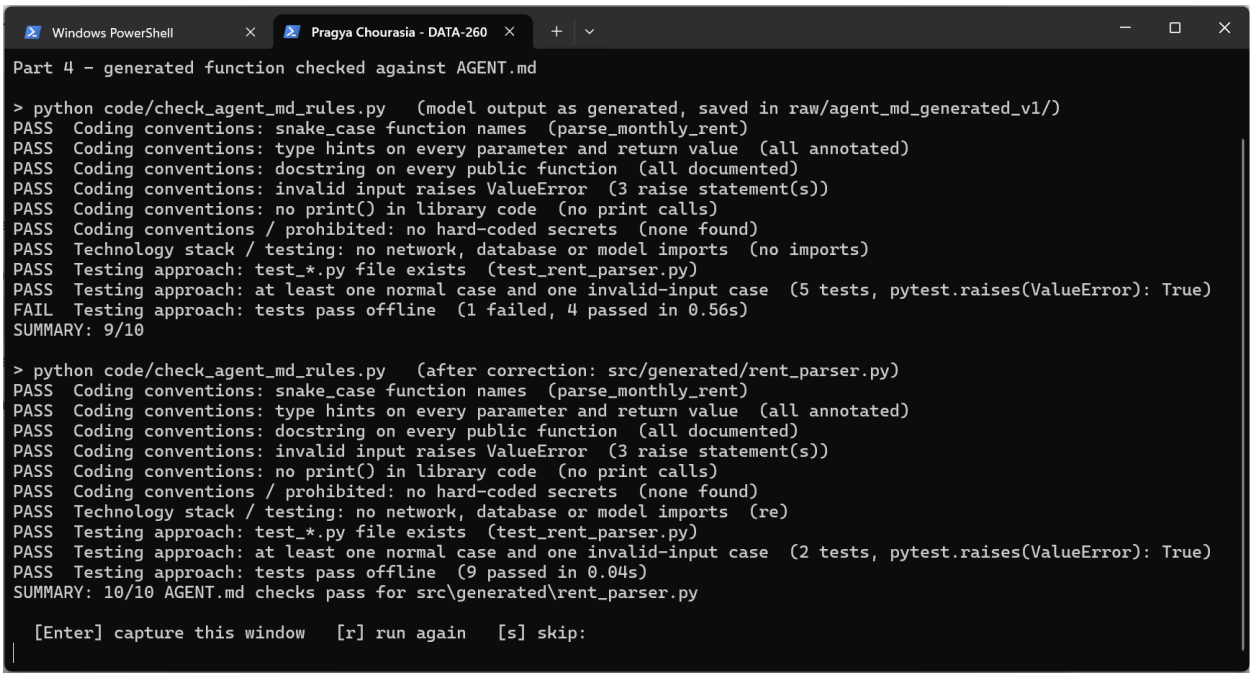
```python
"""Rent-string parsing for listings (generated with AGENT.md, then corrected; see
    reports/hw06)."""
import re

_AMOUNT = re.compile(r"(-?)\s*\$?\s*(\d[\d,]*(?:\.\d+)?)\s*(k\b)?", re.IGNORECASE)


def parse_monthly_rent(text: str) -> int:
    """Convert a listing's rent string into whole US dollars per month.

    Accepts forms such as "$2,400/mo", "2400 per month", "$2.4k" and " $1,950 ".

    Raises:
        ValueError: if the text is empty, contains no amount, or the amount is zero or
            negative.
    """
    if not isinstance(text, str) or not text.strip():
        raise ValueError("rent text is empty")
    match = _AMOUNT.search(text)
    if match is None:
        raise ValueError(f"no rent amount found in {text!r}")
    sign, number, thousands = match.groups()
    amount = float(number.replace(",", ""))
    if thousands:
        amount *= 1000
    if sign:
        amount = -amount
    rent = round(amount)
    if rent <= 0:
        raise ValueError(f"rent must be positive, got {rent}")
    return rent
```



Screenshot 4.3 - Terminal: python code\check_agent_md_rules.py - generated version 9/10, corrected version 10/10

4.6 MongoDB memory vs file-based memory

Criterion	MongoDB memory (Part 2)	File memory (Part 4)
Inspectability	Needs a tool (Compass, mongosh, or GET `/api/memory`). Episodes carry 512-float embeddings that are not human-readable; summaries are LLM-written, so you see the result but not why a fact changed.	Open the file in any editor or run `/memory`. Every fact is one dated line, and consolidation writes its decisions ("Resolved contradictions", "Needs review") into the file. `git diff` shows exactly what changed.
Persistence after a restart	Survives backend and container restarts (named Docker volume `s6758_mongo_data`). Lost only if the volume is deleted. Needs the database running to read it.	Survives anything that keeps the repo; versioned in Git, so earlier states (e.g. `MEMORY_original.md`) can be restored. Readable with no services running.
Scalability across many sessions	Built for it: per-user/per-session queries on indexes, 24 messages + 17 episodes per demo user is trivial, and thousands of users fit easily. Weak spot: retrieval loads every episode for a user and scores cosine in Python, which is O(episodes) per turn; a vector index (Atlas Search / pgvector) would be needed at large scale.	Poor: one shared file, loaded whole into every prompt (about 540 tokens for the 22 consolidated facts), with no per-user separation and no concurrent writers. It must be consolidated to stay under 40 lines, which is exactly what limits it.

Criterion	MongoDB memory (Part 2)	File memory (Part 4)
Facts that must not be forgotten	Risky. Short-term memory slides away, summaries are rewritten by the model (the Part 2 lifetime summary still said $2,400 after the user changed it to $2,700), and episodic retrieval only returns the top-k by similarity, so an important fact can be missed if the wording differs. In the Part 3 N=4 run the "Tesla Model 3" fact scored only 0.02 for "What kind of car do I drive?" because the stemmer turns "drives" into "driv" but leaves "drive" unchanged; the EV-charging fact was retrieved, yet the model still answered that it had no information.	Better. A fact in MEMORY.md is in every prompt, deletions are explicit, contradictions are kept in "Resolved contradictions", and doubtful resolutions are flagged for a human. Still fallible: recency picked the wrong `mongo_db` value until the reviewer pass flagged it.

Use each for what it is good at: MongoDB for high-volume, per-user conversational memory (messages, summaries, episodes) where losing an old detail is acceptable. AGENT.md / MEMORY.md for a small set of durable, reviewed instructions and facts that must always be in context and need a human-readable audit trail.

AI Use

1. What I used an AI assistant for, and what I did myself

I used Claude Code as a coding assistant for most of the implementation in this homework. I gave it the HW6 brief, the two class demos (the FastAPI + PyMongo study planner and the long-term-memory demo) and my HW1-HW5 repository, and used it to plan the work part by part and to write most of the code: the MongoDB listings API (config, connection, schemas and routes in separate files), the memory module behind `/api/chat`, the Part 3 experiment runner and probe scorer, the file-based memory functions and CLI commands, the tests, the verification script, and the layout of this report.

What I did myself: I kept the rental-listings domain from my earlier homeworks and asked for the work to be done one part at a time so I could check and push each part. I set up and ran everything on my own laptop (Docker with MongoDB, the FastAPI backend on port 8458, Ollama with llama3.2:3b), used the chat page myself, ran the demo session and the memory experiments, took the screenshots, and checked the outputs against the requirements in the brief. I decided what to keep when results looked wrong, read the results to understand what the memory settings change, and handled Git (branch, commits, tag).

2. Something the AI produced that was wrong

Three things the assistant wrote were wrong, and each was caught by a test or a check:

- **Embedding stemmer.** The first version of `_stem()` removed "es" before "s", so "drives" became "driv" while "drive" stayed "drive". The stored fact "Pragya drives a Tesla Model 3" only scored 0.02 against the probe "What kind of car do I drive?", so that fact was not retrieved.
- **Function generated under AGENT.md.** I had llama3.2:3b generate `parse_monthly_rent` with AGENT.md as its instructions. It followed all the style rules (snake_case, type hints, docstring, ValueError, no print) but failed its own first example: `"$2,400/mo"` raised a ValueError because the comma and "/mo" were never removed. One of its tests was also wrong (it expected `"$2,400"` to be rejected).
- **Memory consolidation and `/dream`.** Consolidating an already-consolidated MEMORY.md lost the "(seen Nx)" counts and the "Needs review" flags, and `/dream` saved my correction under a new key (`hw6_project_database`) instead of `mongo_db`, so the wrong value was never replaced.

3. How the problems were found

- The stemmer bug showed up in the Part 3 results: at N=4 the car probe scored 1/5. The episodic facts retrieved for that probe (saved in `raw/part3_run1_aborted/n4.json`) and their cosine scores showed the 0.02 score and the "driv" / "drive" mismatch.
- `code/check_agent_md_rules.py` checks each AGENT.md rule separately and also runs the tests. It reported 9/10 for the generated function, with "tests pass offline: 1 failed" (`raw/agent_md_generated_v1/agent_md_check.json`).
- The consolidation bug was caught by the test `test_consolidation_is_idempotent_and_keeps_history`, and the `/dream` problem is visible in the session output `raw/part4_dream_session_before_fix.txt`.

4. What was changed and why it works now

- The stemmer now handles "ies"/"sses"/"ches"/"xes" first and then strips one trailing "s", "ing" or "ed" and a final "e", so "drive", "drives" and "driving" share one stem (the score went from 0.02 to

0.16). Because this changes retrieval, I reran the whole Part 3 sweep so every setting uses the same code. The first run is kept in `raw/part3_run1_aborted/`; it also stopped when MongoDB timed out because other Docker containers on my laptop were using all the memory.

- The generated function was rewritten with one regular expression (optional sign, "$", digits with commas or decimals, optional "k"), and the wrong test was fixed. The checker now reports 10/10 and all tests pass. What I learned: AGENT.md controls how generated code looks, but only tests show whether it actually works.
- Consolidation now reads "(seen Nx)" back as a count, keeps earlier "Needs review" lines until that key is decided again, and `/dream` is given the existing keys and told to reuse them. 14 offline tests cover these cases. The one wrong resolution left (`mongo_db`) I fixed through a recorded human review in MEMORY.md, which is exactly what the "Needs review" section is for.